EXPERIMENT 8

Academic Year: 2026-27 Year and Sem: TE V
Subject: DEV Lab Course Code:

**PROBLEM STATEMENT:**

To implement an automated deployment pipeline using GitHub Actions that automatically builds and deploys a web application whenever changes are pushed to a GitHub repository.

**Requirements:**

Git installed , GitHub account , Internet connection , Visual Studio Code (or any text editor)

**THEORY:**

GitHub Actions is GitHub's built-in CI/CD service.

A **deployment pipeline** automatically performs the following tasks:

Developer

|

Push Code to GitHub

|

GitHub Actions Workflow Starts

|

Build Project

|

Run Tests (Optional)

|

Deploy Website

|

GitHub Pages

**Step 1: Create a Simple Web Application**

Create a folder named:

mkdir github-actions-demo1

cd github-actions-demo1

Inside the folder, create a file:

touch index.html

Add the following code:

```
<!DOCTYPE html>
<html>
<head>
   <title>GitHub Actions Demo1</title>
</head>
<body>
   <h1>Hello from GitHub Actions1!</h1>
   <p>This website is automatically deployed using GitHub Actions.</p>
</body>
</html>
```

Save the file.

Your folder structure should look like:

```
github-actions-demo1/
│
└── index.html
```

start index.html

This will open the webpage in your default browser.

**Step 2: Create a GitHub Repository**

1. Open GitHub.
2. Sign in to your account.
3. Click + → **New repository**.
4. Enter repository name:

github-actions-demo1

5. Select **Public**.
6. Click **Create repository**.

**Step 3: Initialize Git Locally**

Open Terminal.

Navigate to your project folder:

cd github-actions-demo1

Initialize Git:

git init

Add the files:

git add .

Commit the files:

git commit -m "Initial commit"

**Step 4: Connect Local Project to GitHub**

Copy the URL of your GitHub repository.

For example:

git remote add origin https://github.com/YOUR_USERNAME/github-actions-demo.git

Rename the branch to main:

git branch -M main

Push the project:

git push -u origin main

Now your index.html should appear in the GitHub repository.

**Step 5: Enable GitHub Pages**

Go to your GitHub repository.

Select:

**Settings → Pages**

Under **Build and deployment**, select:

Source: GitHub Actions1

This tells GitHub that deployment will be handled using a GitHub Actions workflow.

**Step 6: Create the GitHub Actions Workflow**

Inside your project folder, create the following directories:

mkdir .github

ls -la

Inside .github, create another folder:

mkdir .github/workflows

Inside workflows, create:

Create a file called deploy.yml:

touch .github/workflows/deploy.yml

Therefore, your structure becomes:

github-actions-demo/

│

├── index.html

│

└── .github/

  └── workflows/

    └── deploy.yml

---

**Step 7: Write the GitHub Actions Workflow**

Open deploy.yml and add:

```
name: Deploy Website


on:
  push:
    branches:
      - main


permissions:
  contents: read
  pages: write
  id-token: write
```

```
jobs:
  deploy:
    runs-on: ubuntu-latest

    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}

    steps:
      - name: Checkout Code
        uses: actions/checkout@v4

      - name: Setup Pages
        uses: actions/configure-pages@v5

      - name: Upload Website
        uses: actions/upload-pages-artifact@v3
        with:
          path: .

      - name: Deploy to GitHub Pages
        id: deployment
        uses: actions/deploy-pages@v4
```

**What does this workflow do?**

The workflow performs the following operations:

Developer

↓

Push Code to GitHub

↓

GitHub Actions Triggered

↓

Checkout Source Code

↓

Configure GitHub Pages

↓

Create Deployment Artifact

↓

Deploy Website

↓

Website Available Online

**Step 8: Push the Workflow to GitHub**

Run:

git add .

Then:

git commit -m "Add GitHub Actions deployment"

Push it:

git push

**Step 9: Check GitHub Actions**

Go to your GitHub repository.

Click:

**Actions**

You should see:

Deploy Website

Click the workflow.

You will see the execution steps:

✓ Checkout Code

✓ Setup Pages

✓ Upload Website

✓ Deploy to GitHub Pages

If everything is successful, GitHub will show a green ✓.

**Step 10: Open the Deployed Website**

Go to:

**Repository → Settings → Pages**

GitHub will provide your website URL, typically:

https://YOUR_USERNAME.github.io/github-actions-demo/

Open the URL.

You should see:

**Hello from GitHub Actions!**

**Step 11: Test Automatic Deployment**

Now modify index.html.

For example, change:

<h1>Hello from GitHub Actions!</h1>

to:

<h1>Welcome to Automated Deployment!</h1>

Save the file.

Then execute:

git add .

git commit -m "Update website"

git push

**Step 12: Observe the Automatic Pipeline**

After git push, GitHub Actions automatically starts the workflow.

The process is:

Code Modification

↓

git push

↓

GitHub Repository

↓

GitHub Actions Trigger

↓

Checkout

↓

Build/Package

↓

Deploy

↓

GitHub Pages

↓

Updated Website

Refresh your website.

The updated content should appear automatically.

**Conclusion**

The automated deployment pipeline was successfully implemented using **GitHub Actions**. The experiment demonstrated how code changes pushed to the GitHub repository can automatically trigger a workflow that **checks out the source code, prepares the application, and deploys it to GitHub Pages**.